

Arquitectura de Programación e Integración Hardware para un Sistema de Control de Movimiento XYZ Basado en ESP32 con Interfaz HMI Nextion: Diseño, Implementación y Guía Operativa Completa

Grupo de Trabajo — Seminario Internacional 2025-2
Laboratorio de Automatización · Universidad ECCI

Abstract / Resumen

Resumen — Este trabajo presenta el desarrollo completo de la arquitectura de programación, la guía de integración hardware y el manual operativo de usuario para un sistema de control de movimiento de tres ejes lineales (X–Y–Z) basado en microcontroladores ESP32 programados en MicroPython. El sistema integra: generación de señales STEP/DIR para motores paso a paso NEMA con encoder en lazo cerrado mediante drivers industriales CL57T, comunicación maestro–esclavo I²C, adquisición de temperatura PT100 vía transmisores 4–20 mA, servidor web embebido para supervisión remota, e interfaz gráfica HMI Nextion con cinco módulos funcionales. Se documentan exhaustivamente los procedimientos de cableado, configuración de DIP-switches, secuencia segura de encendido, calibración, pruebas y diagnóstico. El sistema fue validado en una plataforma real de transporte lineal didáctico, logrando control preciso de posición, detección de límites en menos de 100 ms y monitoreo en tiempo real desde cualquier dispositivo de la red local.

Palabras clave — ESP32, ESP32-S3, MicroPython, control de movimiento, motores paso a paso, driver CL57T, lazo cerrado, I²C maestro–esclavo, PT100, Nextion HMI, automatización industrial, servidor web embebido, calibración, sistema de posicionamiento XYZ.

TABLA DE CONTENIDOS

I.	Introducción	2
II.	Arquitectura General del Sistema	2
III.	Módulo de Comunicación I ² C Maestro–Esclavo	3
IV.	Módulo de Sensores PT100	3
V.	Módulo de Finales de Carrera	4
VI.	Módulo WiFi y Servidor Web	4
VII.	Interfaz HMI Nextion	4
VIII.	Gestión de Timers y Bucle Principal	6
IX.	Guía de Integración Hardware	6
X.	Guía Operativa del Sistema HMI XYZ	9
XI.	Resultados y Validación	11
XII.	Discusión	12
XIII.	Conclusiones	12
XIV.	Referencias	13

I. INTRODUCCIÓN

Los sistemas de control de movimiento multi-eje constituyen el núcleo operativo de numerosas aplicaciones industriales: manufactura CNC, robótica cartesiana, equipos de inspección automatizada y plataformas de transporte interno [5]. La demanda de precisión, repetibilidad y capacidad de diagnóstico en tiempo real ha impulsado el desarrollo de arquitecturas que combinan microcontroladores de bajo costo con protocolos de comunicación industrial robustos y drivers de potencia inteligentes. En este contexto, la adopción de plataformas como el ESP32 —con capacidades WiFi, I²C, UART y ADC integradas— representa una alternativa viable y económica frente a soluciones PLC de alto costo para entornos educativos y de I+D.

El presente trabajo describe íntegramente el sistema de control CNC – UNI. ECCI: un módulo de transporte lineal de tres ejes (X–Y–Z) desarrollado en el Laboratorio de Automatización de la Universidad ECCI. El sistema emplea motores paso a paso NEMA

con encoder integrado, gobernados por drivers industriales CL57T en lazo cerrado, controlados mediante señales STEP/DIR generadas por un ESP32-S3 como nodo esclavo I²C. Un ESP32 maestro gestiona la capa de supervisión: interfaz HMI Nextion, servidor web embebido, adquisición de temperatura PT100 y lectura de finales de carrera.

A diferencia de trabajos previos, este documento integra en un único cuerpo técnico: (i) la arquitectura completa de software y hardware, (ii) el conjunto exhaustivo de comandos ASCII, (iii) la guía detallada de cableado y verificación, y (iv) la guía operativa de usuario para el operador de planta. Esta integración facilita tanto la replicación por nuevos integrantes como la transferencia del sistema a entornos de producción didáctica.

Contribuciones principales:

- Arquitectura maestro–esclavo I²C para distribución del control de movimiento entre dos ESP32.
- Integración de drivers CL57T en lazo cerrado con motores NEMA con encoder.
- Cinco pantallas HMI Nextion especializadas con lógica de navegación y seguridad.
- Servidor web embebido con actualización automática de sensores cada 2 s.
- Conjunto completo de comandos ASCII para control CNC desde HMI y red.
- Guía de integración hardware con procedimientos de verificación y diagnóstico.
- Manual operativo de usuario integrado como sección del documento técnico.
- Validación en plataforma real con motores paso a paso y sensores de límite.

II. ARQUITECTURA GENERAL DEL SISTEMA

II-A. Descripción Hardware

La Tabla I resume los componentes principales. La arquitectura se organiza en tres capas: (1) supervisión —ESP32 maestro, HMI Nextion, PT100, finales de carrera—; (2) control de movimiento —ESP32-S3 esclavo, drivers CL57T, motores NEMA con encoder—; (3) potencia —fuente 24–48 VDC, transceptor RS-485 para VFD—.

Cuadro I: Componentes Hardware del Sistema

Componente	Descripción
ESP32 (Maestro)	Microcontrolador principal; ejecuta MicroPython, gestiona HMI, WiFi e I ² C
ESP32-S3 N16R8 (Esclavo)	Controlador de bajo nivel; genera señales STEP/DIR para motores
Pantalla Nextion	HMI gráfica con 5 páginas; comunicación UART a 9600 bps
Drivers CL57T ×3	Drivers industriales lazo cerrado para ejes X, Y, Z
Motores NEMA + Encoder ×3	Motores paso a paso con encoder integrado para closed-loop
Sensores PT100 ×3	Transmisores 4–20 mA; resistencia shunt 165 Ω
Finales de carrera ×12	4 por eje: límite inferior, fallo inf., HOME, fallo superior
Transceptor RS-485	Comunicación industrial hacia VFD externo
Fuente 24–48 VDC / 10 A	Alimentación de potencia para los tres drivers CL57T

II-B. Arquitectura de Software

El software se organiza en módulos funcionales desacoplados. La comunicación entre módulos se realiza mediante variables globales y colas de comandos asíncronos, evitando condiciones de carrera en el bucle cooperativo de MicroPython.

- **Módulo WiFi/AP:** Crea punto de acceso SSID ESP32_Master; máx. 2 clientes simultáneos.
- **Módulo I²C Maestro:** Gestiona comunicación con el esclavo; envío síncrono y asíncrono.
- **Módulo Nextion:** Envía/recibe comandos UART; detecta cambios de página y botones.
- **Módulo PT100:** Lee tres canales ADC; convierte señal 4–20 mA a temperatura °C.
- **Módulo Sensores:** Lee 12 finales de carrera distribuidos en tres ejes.
- **Módulo Servidor Web:** Página HTML con actualización automática y endpoint /i2c.
- **Módulo Comandos CNC:** Funciones de alto nivel que encapsulan comandos ASCII al esclavo.
- **Gestor de Timers:** Tres temporizadores hardware coordinan tareas periódicas.

III. MÓDULO DE COMUNICACIÓN I²C MAESTRO-ESCLAVO

III-A. Configuración del Bus I²C

El ESP32 maestro inicializa el bus I²C en GPIO 21 (SDA) y GPIO 22 (SCL) a 100 kHz. La dirección del esclavo es 0x08. Al inicio se ejecuta un escaneo para verificar la presencia del dispositivo esclavo antes de habilitar el sistema de control.

```
from machine import I2C, Pin
I2C_MASTER_SDA = 21; I2C_MASTER_SCL = 22
I2C_SLAVE_ADDRESS = 0x08
def init_i2c_master():
    global i2c
    i2c = I2C(0, scl=Pin(I2C_MASTER_SCL),
              sda=Pin(I2C_MASTER_SDA), freq=100000)
    devices = i2c.scan()
    if I2C_SLAVE_ADDRESS in devices:
        print("Esclavo I2C conectado")
        return True
    return False
```

Listing 1: Inicialización I²C Maestro

III-B. Protocolo de Comandos ASCII

Los comandos se transmiten como cadenas ASCII. Este diseño simplifica la depuración y permite enviar comandos desde la HMI, el servidor web o el monitor serial de forma intercambiable.

Cuadro II: Conjunto de Comandos CNC vía I²C

Comando	Descripción
HOME	Inicia rutina de homing para todos los ejes
STOP	Parada de emergencia inmediata
RESET	Restablece el sistema al estado inicial
RUTINA_1 / RUTINA_2	Ejecuta secuencia automática 1 o 2
GET_POSITION	Solicita posición actual X, Y, Z
STATUS	Solicita estado operativo del esclavo
START_JOG eje dir vel	Inicia movimiento JOG continuo
STOP_JOG	Detiene el movimiento JOG
SET_X_MM n	Configura dimensión X en mm
SET_DEBOUNCE n	Configura tiempo de debounce en ms
PING	Prueba de conectividad I ² C

IV. MÓDULO DE SENSORES PT100

Cada sensor PT100 está conectado a un transmisor industrial 4–20 mA. La corriente circula a través de un resistor shunt de 165 Ω generando voltaje entre 0,66 V (4 mA = 0 °C) y 3,3 V (20 mA = 200 °C). Los pines ADC utilizados son GPIO 32, 33 y 34 con atenuación 11 dB y resolución de 12 bits. La conversión aplica transformación lineal en dos etapas:

$$I_{mA} = V_{ADC} / R_{shunt} \times 1000 \quad (1)$$

$$T[°C] = T_{min} + (I_{mA} - 4) \times (T_{max} - T_{min}) / 16 \quad (2)$$

```
def leer_pt100(adc_sensor, sensor_num):
    adc_value = adc_sensor.read()          # 0 - 4095
    voltage = (adc_value / 4095.0) * 3.3   # Voltios
    current_ma = (voltage / 165.0) * 1000 # mA
    current_ma = max(4.0, min(20.0, current_ma))
    temperatura = 0.0 + ((current_ma - 4.0) * 200.0 / 16.0)
    return adc_value, voltage, temperatura
```

Listing 2: Función de lectura PT100

V. MÓDULO DE FINALES DE CARRERA

Cada eje cuenta con cuatro señales digitales para detectar límites de seguridad y la posición HOME. Al detectar la activación de cualquier límite se envía inmediatamente el comando STOP al esclavo y se actualiza la pantalla Nextion.

Cuadro III: Asignación de Pines – Finales de Carrera (ESP32 Maestro)

Señal	Eje X	Eje Y	Eje Z
Límite inferior	GPIO 13	GPIO 26	GPIO 4
Fallo límite inf.	GPIO 12	GPIO 25	GPIO 18
Sensor HOME	GPIO 14	GPIO 15	GPIO 16
Fallo límite sup.	GPIO 27	GPIO 2	GPIO 19

```
def leer_switches():
    limite_x = estados_switches_x[0] or estados_switches_x[3]
    limite_y = estados_switches_y[0] or estados_switches_y[3]
    limite_z = estados_switches_z[0] or estados_switches_z[3]
    if (limite_x or limite_y or limite_z) and sistema_activo:
        send_i2c_command("STOP")
        send_cmd('t14.txt="!LIMITE ACTIVADO!"')
```

Listing 3: Detección de límites y parada de emergencia

VI. MÓDULO WIFI Y SERVIDOR WEB

El ESP32 crea un punto de acceso inalámbrico con SSID **ESP32_Master** y contraseña **12345678**, limitado a 2 clientes simultáneos. El servidor HTTP sirve una página con actualización automática cada 2 s mostrando temperaturas PT100, estado de los 12 finales de carrera, estado I²C y botones de control CNC. El endpoint **/i2c?command=CMD** permite enviar comandos directamente desde el navegador sin software adicional.

VII. INTERFAZ HMI NEXTION

VII-A. Descripción General

La interfaz se implementa en una pantalla Nextion programada con Nextion Editor. La comunicación se realiza mediante UART1 del ESP32 (TX: GPIO 17, RX: GPIO 16) a 9600 bps con terminador 0xFF 0xFF 0xFF. Se diseñaron cinco pantallas especializadas con esquema de color verde/amarillo/rojo alineado con IEC 60073 [6].

```
uart = UART(1, 9600, tx=Pin(17), rx=Pin(16))
def send_cmd(cmd):
    uart.write(cmd.encode('utf-8'))
    uart.write(b'\xFF\xFF\xFF')
```

Listing 4: Función de envío de comandos a Nextion

Cuadro IV: Resumen de Pantallas HMI Nextion

Pantalla	Nombre	Función Principal
P0	Supervisión CNC	Posición X/Y/Z, PT100, estado sistema, acciones rápidas HOME/STOP/RESET
P1	Calibración	Offset X/Y/Z con botones +/-, homing individual por eje, guardar parámetros
P2	Pruebas y Diagnóstico	Test individual por eje y sensor, registro de resultados, barra de progreso
P3	Monitoreo	Visualización continua RUNNING/STOP/ERROR, dial de velocidad, PT100, VFD
P4	Asistencia	Chat bot IA, descarga de guía PDF, estado de conexión ESP32

VII-B. Pantalla Principal – Supervisión CNC

La pantalla principal centraliza la información operativa organizada en cuatro paneles: (i) Posición de Ejes con coordenadas X/Y/Z en tiempo real, (ii) Lectura de Sensores con temperatura PT100 y frecuencia VFD, (iii) Estado del Sistema con alertas activas, y (iv) Acciones Rápidas con botones HOME, STOP y RESET.



Fig. 1. Pantalla principal del sistema CNC – UNI. ECCI: paneles de posición, sensores, estado del sistema y acciones rápidas.

VII-C. Pantalla de Calibración del Sistema

Permite configurar los parámetros de posicionamiento de cada eje y ejecutar la rutina de homing. Los valores ingresados se envían al ESP32 maestro que los traduce a comandos I²C hacia el esclavo (SET_X_MM, SET_DEBOUNCE, SET_HOME_OFFSET). Los offsets se persisten en **calib.txt**.



Fig. 2. Pantalla de calibración del sistema: ajuste de offset por eje X, Y, Z e inicio de homing individual.

VII-D. Pantalla de Pruebas y Diagnóstico

Proporciona herramientas de diagnóstico para verificar el correcto funcionamiento de todos los subsistemas antes de la operación en producción. Se divide en cuatro secciones: Pruebas de Movimiento (TEST EJE X/Y/Z, TEST COM), Prueba de Sensores (Límites Fijos, Límites Móviles, Temperatura, VFD RS-485), Registro de Resultados y Estado Actual con barra de progreso.



Fig. 3. Pantalla de pruebas y diagnóstico: pruebas de movimiento por eje, verificación de sensores y registro de resultados.

VII-E. Pantalla de Monitoreo

Opera en modo de visualización continua mostrando: posición X/Y/Z actualizada desde el esclavo I²C, lecturas PT100, estado del motor y frecuencia VFD. Tres indicadores de color (RUNNING–verde, STOP–amarillo, ERROR–rojo) reflejan el estado operativo del esclavo conforme a IEC 60073.



Fig. 4. Pantalla de monitoreo: visualización en tiempo real de posición, sensores y estado del sistema (RUNNING / STOP / ERROR).

VII-F. Pantalla de Asistencia al Usuario

Proporciona información de soporte: nombre del proyecto, versión v1.0, descripción del sistema, chat con bot de IA para preguntas frecuentes, descarga directa de la Guía de Usuario PDF y estado de conexión del ESP32.



Fig. 5. Pantalla de asistencia: información del sistema, chat con bot de ayuda, descarga de guía y estado de conexión del ESP32.

VIII. GESTIÓN DE TIMERS Y BUCLE PRINCIPAL

El sistema utiliza tres temporizadores de hardware para coordinar las tareas de forma no bloqueante. El planificador cooperativo opera con período base de 100 ms.

Cuadro V: Frecuencias de Actualización de Tareas

Tarea	Período	Prioridad
Procesar comandos Nextion	Cada ciclo (100 ms)	Alta
Leer finales de carrera	Cada ciclo (100 ms)	Alta
Verificar conexión I ² C	Cada ciclo (100 ms)	Alta
Leer sensores PT100	Cada 0,5 s	Media
Enviar datos I ² C al esclavo	Cada 1 s	Media
Leer mensajes del esclavo	Cada 2 s	Media
Actualizar pantalla Nextion	Cada 4 s	Baja

```
def tarea_principal_combinada(timer):
    if inicializacion_completada:
        procesar_comandos_nextion() # cada ciclo
        leer_switches() # cada ciclo
        verificar_conexion_i2c() # cada ciclo
        if contador % 5 == 0:
            leer_todos_pt100() # cada 0.5 s
        if contador % 20 == 0:
            leer_mensajes_esclavo() # cada 2 s
        if sistema_activo and contador % 10 == 0:
            enviar_datos_i2c() # cada 1 s
        if contador >= 40:
            actualizar_pantalla() # cada 4 s
            contador = 0
```

Listing 5: Planificador de tareas cooperativo

IX. GUÍA DE INTEGRACIÓN HARDWARE

Esta sección documenta el procedimiento completo de integración hardware para el ESP32-S3 N16R8 (esclavo I²C) que gobierna directamente los drivers CL57T.

IX-A. Advertencias y Buenas Prácticas Críticas

■ **IMPORTANTE:** Nunca conectar o desconectar elementos con el sistema energizado. Toda intervención debe realizarse con las fuentes apagadas para evitar daños al ESP32 y a los drivers CL57T.

Conexión sin alimentación: Nunca conectar/desconectar elementos energizados.

Verificación previa de voltajes: ESP32 alimentado por USB (5 V) o 3,3 V; drivers entre 24–48 VDC. Lógica S3 en 5 V.

Referencia común de masa: ESP32 y los tres drivers CL57T deben compartir un mismo punto GND.

Orden de encendido: Primero energizar drivers (24–48 VDC); luego conectar ESP32.

IX-B. Lista de Materiales

Cuadro VI: Materiales Eléctricos y Mecánicos Requeridos

Cant.	Descripción
1	Microcontrolador ESP32-S3 N16R8
3	Driver paso a paso CL57T V4.0
3	Motor NEMA paso a paso con encoder integrado (17, 23 o 24)
1	Fuente de alimentación 24–48 VDC, 10 A mínimo
1	Fuente 5 V para ESP32 (USB o regulador externo)
6	Sensores de límite / finales de carrera (contacto NO)
—	Cable apantallado para señales de control
—	Cable de potencia AWG 18 o superior
—	Borneras, conectores y fusibles individuales (5 A por driver)

IX-C. Configuración de los Drivers CL57T

El switch S3 **debe** fijarse en la posición 5 V obligatoriamente. Si se mantiene en 24 V con el ESP32 conectado directamente, el microcontrolador resultará dañado de forma irreversible.

Cuadro VII: Configuración de DIP-switches del Driver CL57T

SW1	SW2	SW3	SW4	SW5	SW6	SW7	SW8
OFF	OFF	ON	ON	OFF	OFF	OFF	OFF

SW1–SW4: microstepping 1/16 (3200 pasos/vuelta). SW6: lazo cerrado activo. SW7: modo PUL/DIR. SW8: filtro 1,5 ms.

IX-D. Señales de Control ESP32-S3 → Drivers

```
// Pines de control de motores (ESP32-S3 esclavo)
const int X_STEP = 20, X_DIR = 21;
const int Y_STEP = 48, Y_DIR = 42;
const int Z_STEP = 37, Z_DIR = 38;
const int Enable = 47; // compartido X, Y, Z
```

Listing 6: Definición de pines de control en el ESP32-S3 esclavo

IX-E. Asignación de Sensores de Límite al ESP32-S3

Cuadro VIII: Sensores de Límite al ESP32-S3 (Esclavo)

Sensor	GPIO	Modo	Ubicación
X H0	3	INPUT_PULLUP	Límite inicial X
X HF	46	INPUT_PULLUP	Límite final X
Y H0	5	INPUT_PULLUP	Límite inicial Y
Y HF	10	INPUT_PULLUP	Límite final Y
Z H0	18	INPUT_PULLUP	Límite inicial Z
Z HF	4	INPUT_PULLUP	Límite final Z

■ **NOTA:** GPIO 46 en algunos módulos ESP32-S3 está reservado. Si presenta comportamiento anómalo, sustituir por GPIO 1, 2, 6, 7, 11–17.

IX-F. Tabla Consolidada de Conexiones

Cuadro IX: Tabla Resumen de Conexiones del Sistema

Origen	Destino	Función
GPIO 20 (ESP32-S3)	CL57T-X PUL+	Pulsos STEP eje X
GPIO 21 (ESP32-S3)	CL57T-X DIR+	Dirección eje X
GPIO 48 (ESP32-S3)	CL57T-Y PUL+	Pulsos STEP eje Y
GPIO 42 (ESP32-S3)	CL57T-Y DIR+	Dirección eje Y
GPIO 37 (ESP32-S3)	CL57T-Z PUL+	Pulsos STEP eje Z
GPIO 38 (ESP32-S3)	CL57T-Z DIR+	Dirección eje Z
GPIO 47 (ESP32-S3)	ENA+ X, Y, Z	Habilitación compartida
GND (todos)	PUL-/DIR-/ENA-	Referencia común
GPIO 8 (ESP32-S3)	SDA bus maestro	Datos I²C
GPIO 9 (ESP32-S3)	SCL bus maestro	Reloj I²C
+24–48 V	Drivers P4 (paralelo)	Alimentación de potencia

IX-G. Secuencia de Encendido Segura

1. Verificación previa sin alimentación: continuidad del GND común, ausencia de cortocircuitos y posición correcta de DIP-switches.
2. Energización de drivers únicamente: encender fuente 24–48 VDC y comprobar LED verde en los tres drivers.
3. Conexión del ESP32: conectar USB, cargar el código y observar mensajes en el monitor serial.
4. Verificación de señales: medir nivel lógico del pin Enable (GPIO 47, debe ser HIGH $\approx 3,3$ V).
5. Prueba de movimiento controlado: enviar START_JOG X 1 1000 y luego STOP_JOG.

IX-H. Solución de Problemas Comunes

Cuadro X: Diagnóstico y Solución de Problemas

Síntoma	Posible Causa	Solución
Motor no se mueve	LED driver apagado; Enable bajo; DIP incorrecto	Verificar LED verde, nivel GPIO 47, configuración S2/S3
Motor vibra sin girar	Bobinas mal cableadas o microstepping excesivo	Intercambiar A+/A-; reducir microstepping
Pérdida de pasos	Corriente insuficiente o aceleración agresiva	Aumentar selector S1; reducir velocidad máxima
Sensores no detectan	GPIO 46 conflicto; contacto NC en lugar de NO	Reasignar GPIO; verificar tipo de contacto
Driver alarma ×1 parpadeo	Sobrecorriente / cortocircuito	Revisar cableado P3 del motor
Driver alarma ×2 parpadeos	Sobretensión en la fuente	Reducir voltaje de alimentación
Driver alarma ×4 parpadeos	Motor no detectado	Revisar conexión P3 y bobinas
Driver alarma ×7 parpadeos	Error de posición del encoder	Revisar conexión P2 del encoder
Reinicios ESP32	Picos de corriente al arrancar motores	Capacitor 1000 μ F; fuente 5 V externa

IX-I. Lista de Verificación Final

Antes de la primera puesta en marcha confirmar:

- Switch S3 de los tres drivers en posición 5 V.
- Switch SW6 de los tres drivers en OFF (lazo cerrado activo).
- Switch SW7 de los tres drivers en OFF (modo PUL/DIR).
- Plano de masa común entre ESP32 y los tres drivers.
- Alimentación 24–48 V conectada en paralelo a los puertos P4.
- Motores conectados al puerto P3 con continuidad de bobinas verificada.
- Encoders conectados al puerto P2 de cada driver.
- Seis sensores de límite cableados con pull-up activo.

- Código del esclavo cargado en el ESP32-S3.
- Monitor serial abierto a 115200 baudios.

X. GUÍA OPERATIVA DEL SISTEMA HMI XYZ

Esta sección integra el manual operativo del sistema HMI XYZ destinado al operador de planta. Describe el procedimiento de inicio, navegación entre módulos, calibración, pruebas, monitoreo, control y finalización segura del sistema.

X-A. Inicio del Sistema

El sistema carga automáticamente los offsets de calibración desde **calib.txt** al encender. No es necesario re-calibrar en cada inicio si los parámetros ya fueron guardados previamente.

Flujo de inicio recomendado:

1. Energizar ESP32 y pantalla Nextion.
2. Verificar que la pantalla principal cargue sin errores.
3. Presionar el botón **HOME** en la pantalla principal.
4. Verificar que los ejes X, Y y Z inicien el movimiento de referenciado.
5. Revisar temperaturas PT100 y estado general del sistema.

■ **NOTA:** El sistema carga automáticamente los offsets de calibración guardados desde el archivo **calib.txt** al encender.

X-B. Navegación General

Cuadro XI: Módulos del Sistema HMI XYZ

Pantalla	Módulo	Descripción
P0	Pantalla Principal	Estado general, ejes, temperaturas y acciones rápidas
P1	Calibración	Ajuste de offsets X, Y, Z y guardado de parámetros
P2	Pruebas	Tests individuales y prueba general del sistema
P3	Monitoreo	Visualización en tiempo real de ejes, temperatura y motor
P4	Asistencia	Guía de operación y estado de comunicación ESP32
P5	Guía QR	Acceso a documentación externa mediante código QR

X-C. Procedimiento de Calibración

El rango de offset permitido es de -50 a +50 mm por eje. Los offsets se persisten en el archivo **calib.txt** en la memoria interna del ESP32.

Procedimiento paso a paso:

1. Presionar el botón **CALIBRACIÓN** en la pantalla principal.
2. Observar los valores actuales de posición para X, Y y Z.
3. Usar los botones + / - de cada eje para ajustar el offset (rango: -50 a +50).
4. Presionar **Iniciar Calibración** para aplicar los offsets en la sesión actual.
5. Presionar **Guardar Parámetros** para persistir los valores en memoria (**calib.txt**).
6. Verificar los valores actualizados en pantalla.
7. Presionar **BACK** para regresar a la pantalla principal.

■ **IMPORTANTE:** Iniciar Calibración y Guardar Parámetros son dos acciones separadas. Si solo presiona Iniciar Calibración sin guardar, los valores se perderán al apagar.

Cuadro XII: Parámetros de Calibración

Parámetro	Valor
Rango de offset permitido	-50 a +50 (por eje)

Archivo de persistencia	calib.txt (memoria interna ESP32)
HOME X / Y / Z	Lleva el eje individual a 0,00 mm
CANCEL	Restaura offsets y valores guardados al entrar al módulo

X-D. Procedimiento de Pruebas

Cuadro XIII: Tests Disponibles en el Módulo de Pruebas

Test	Descripción
Test Eje X	Verifica posición, temperatura y sensores de límite del eje X
Test Eje Y	Verifica posición, temperatura y sensores de límite del eje Y
Test Eje Z	Verifica posición, temperatura y sensores de límite del eje Z
Límites Fijos	Valida rangos de operación (0–330 / 0–200 / 0–100 mm)
Límites Móviles	Verifica que la posición actual de cada eje esté dentro de su rango
Temperatura	Comprueba que las tres temperaturas estén entre 20 °C y 45 °C
Prueba General	Ejecuta todos los tests anteriores en secuencia automática

■ **NOTA:** Cada test individual puede ejecutarse de forma independiente. El botón STOP puede usarse en cualquier momento para detener el proceso de forma segura.

X-E. Monitoreo del Sistema

Cuadro XIV: Botones de Control – Pantalla de Monitoreo

Botón	Acción
RUNNING	Reanuda el movimiento de ejes; limpia condición de error o parada
STOP	Parada suave (soft stop); conserva la posición de los ejes
ERROR / RESET	Reinicia completamente todas las variables del sistema a cero
REFRESH	Fuerza una actualización inmediata de los valores en pantalla
CLEAR	Borra los valores mostrados sin afectar el sistema
BACK	Regresa a la pantalla principal

■ **IMPORTANTE:** El botón STOP en Monitoreo realiza parada suave (soft stop). El botón STOP de la Pantalla Principal ejecuta parada de emergencia completa. Son comportamientos distintos.

X-F. Rangos de Operación

Cuadro XV: Rangos de Operación del Sistema

Parámetro	Valor
Eje X — rango permitido	0,00 mm a 330,00 mm
Eje Y — rango permitido	0,00 mm a 200,00 mm
Eje Z — rango permitido	0,00 mm a 100,00 mm
Temperatura normal	20,0 °C a 45,0 °C
Rango de offset de calibración	–50 a +50 (por eje)
Velocidad simulada X / Y / Z	1,8 / 1,2 / 0,7 mm por ciclo (valor inicial)

X-G. Acciones de Control Rápidas

Cuadro XVI: Acciones de Control de la Pantalla Principal

Botón	Acción	Efecto en el sistema
HOME	Referenciado de ejes	Mueve X, Y, Z a 0,00 mm gradualmente e inicia movimiento simulado
STOP	Parada de emergencia	Detiene todo movimiento; activa estado de emergencia y congela temperaturas

RESET	Reinicio completo	Restablece todas las variables a valores iniciales; requiere HOME posterior
-------	-------------------	---

X-H. Finalización Segura del Sistema

1. Verificar que no haya ninguna prueba o proceso en ejecución.
2. Presionar **STOP** en la pantalla principal para detener el movimiento de ejes.
3. Confirmar que el estado del sistema indique STOP o *Sistema listo*.
4. Regresar a la pantalla principal si se encuentra en otro módulo.
5. Apagar el sistema de forma controlada desconectando la alimentación.

■ **IMPORTANTE:** No desconectar la alimentación durante una prueba en ejecución o durante el proceso de HOME, ya que podría corromper el archivo calib.txt.

XI. RESULTADOS Y VALIDACIÓN

La implementación fue validada sobre una plataforma de transporte lineal real con motores NEMA 23 con encoder de 1000 ppr, drivers CL57T V4.0 configurados a 3200 pasos/vuelta y ESP32 maestro corriendo MicroPython v1.21.

Cuadro XVII: Métricas de Desempeño del Sistema

Métrica	Valor Obtenido	Observación
Latencia detección de límites	< 100 ms	Lectura en cada ciclo del planificador
Resolución de temperatura	0,1 °C	ADC 12 bits + shunt 165 Ω calibrado
Rango de temperatura monitoreado	0 – 200 °C	Transmisor PT100 4–20 mA
Frecuencia de actualización web	2 s	Servidor HTTP no bloqueante
Actualización pantalla Nextion	4 s	Planificador cooperativo
Clientes WiFi simultáneos	2 máx.	Optimización de memoria RAM
Pines GPIO utilizados (maestro)	19	Ver Cuadro XVIII
Tamaño archivo principal (Flash)	~46 KB	MicroPython compilado
Pasos por vuelta (microstepping)	3200	Configuración SW1–SW4 CL57T
Resultado HOME automático	Exitoso en 3 ejes	Detección sensor H0 confirmada

Cuadro XVIII: Uso de Recursos del ESP32 Maestro

Recurso	Valor
Pines GPIO utilizados	19
Canales UART	2 (Nextion + RS-485)
Canal I ² C	1 (maestro, 100 kHz)
Canales ADC	3 (PT100)
Timers hardware	3
Memoria Flash (archivo principal)	~46 KB
Punto de acceso WiFi	Sí (máx. 2 clientes)

XII. DISCUSIÓN

La arquitectura maestro–esclavo I²C demostró ser efectiva para separar la capa de supervisión (ESP32 maestro: HMI, WiFi, sensores analógicos) de la capa de control de movimiento (ESP32-S3 esclavo: generación de pulsos STEP/DIR). Esta separación mejora el determinismo temporal del control de motores al aislar las tareas de supervisión del lazo de control de bajo nivel. La frecuencia de bus de 100 kHz resultó adecuada para el volumen de comandos ASCII del protocolo, con latencias de transmisión por debajo de 5 ms para comandos de hasta 20 caracteres.

La integración de los drivers CL57T en lazo cerrado con motores NEMA dotados de encoder aporta una mejora sustancial frente a configuraciones en lazo abierto: el propio driver detecta y corrige automáticamente las pérdidas de paso, lo que se traduce en mayor robustez frente a perturbaciones mecánicas y mayores velocidades operativas sin sacrificar la precisión de posicionamiento. Esta característica es particularmente relevante en el contexto didáctico, donde las cargas variables y los arranques frecuentes pueden causar pérdida de pasos en sistemas de lazo abierto convencionales.

El diseño de las cinco pantallas Nextion siguió una lógica de flujo operativo clara alineada con IEC 60073 [6]. La integración del manual operativo de usuario en el cuerpo técnico del documento elimina la brecha entre documentación técnica y procedimientos de operación, garantizando coherencia cuando el sistema evoluciona.

Trabajo futuro: Se propone la implementación de Modbus RTU sobre RS-485 para integración con PLCs, la incorporación de un gemelo digital mediante MQTT, la expansión del bot de asistencia con lógica de diagnóstico automatizado, y la migración del firmware del esclavo a FreeRTOS sobre ESP-IDF para garantizar determinismo en generación de pulsos a frecuencias superiores a 50 kHz.

XIII. CONCLUSIONES

1. La arquitectura maestro-esclavo I²C permite distribuir la carga computacional y aislar las capas de supervisión y control, mejorando la robustez del sistema.
2. El conjunto de comandos ASCII sobre I²C proporciona una interfaz simple y extensible integrable desde la HMI Nextion, el servidor web o el monitor serial.
3. La interfaz HMI Nextion de cinco pantallas cubre todos los modos operativos con retroalimentación visual clara mediante el esquema de color verde/amarillo/rojo conforme a IEC 60073.
4. El servidor web embebido extiende la supervisión a cualquier dispositivo con navegador dentro de la red local, sin requerir software adicional.
5. La adopción de drivers CL57T en lazo cerrado con motores NEMA con encoder eleva la confiabilidad del subsistema de movimiento al eliminar la pérdida de pasos de las configuraciones en lazo abierto.
6. La guía de integración hardware documentada, incluyendo DIP-switches, secuencia de encendido y rutinas de verificación, constituye un activo reutilizable para futuras iteraciones del proyecto.
7. La integración del manual operativo en el cuerpo técnico del documento elimina la brecha entre documentación técnica y procedimientos de operación, garantizando coherencia.
8. MicroPython demostró ser adecuado para desarrollo rápido de sistemas embebidos de complejidad media con requisitos de tiempo real moderados.

XIV. REFERENCIAS

- [1] Espressif Systems, "ESP32 Technical Reference Manual," Version 5.1, 2023. Disponible: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [2] ITEAD Studio, "Nextion Instruction Set," Version 1.65, 2022. Disponible: <https://nextion.tech/instruction-set/>
- [3] Modbus Organization, "Modbus Application Protocol Specification V1.1b3," 2012. Disponible: https://modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf
- [4] G. van Rossum y MicroPython Contributors, "MicroPython – Python for Microcontrollers," 2023. Disponible: <https://micropython.org/>
- [5] N. S. Nise, Control Systems Engineering, 6th ed. Hoboken, NJ: Wiley, 2011.
- [6] International Electrotechnical Commission, "IEC 60073: Basic and safety principles for man-machine interface," 2002.
- [7] Leadshine Technology, "CL57T Closed Loop Stepper Driver User Manual," V4.0, 2022.
- [8] Integrador20252, "HMI esp32," GitHub, 2025. Disponible: https://github.com/Integrador20252/HMI_esp32
- [9] Universidad ECCI, "Guía de Usuario del Sistema HMI XYZ," v1.0, Laboratorio de Automatización, Bogotá, Colombia, 2025.